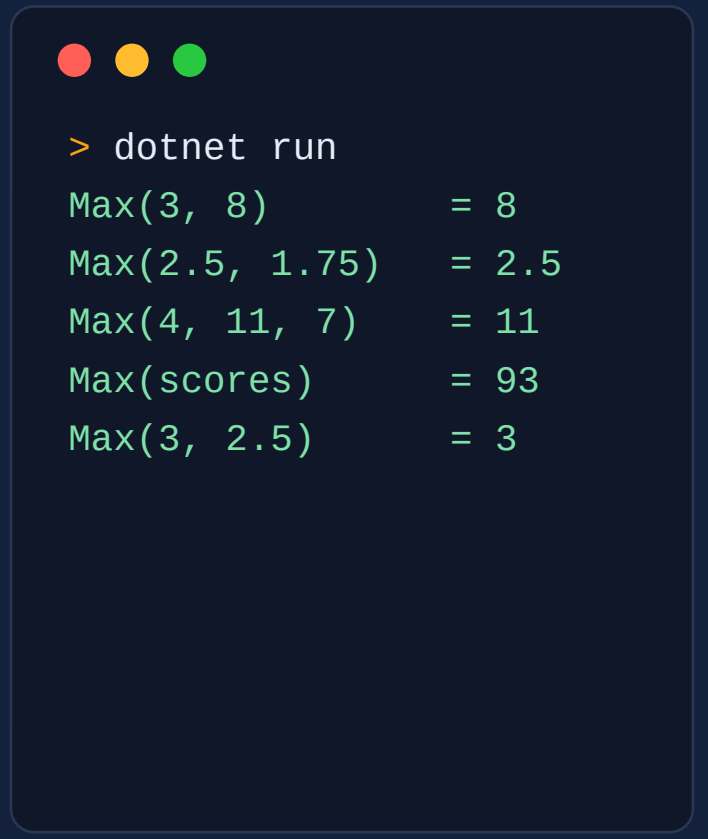
>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 12

Functions (Methods): Definition, Calling, Parameter Passing

Why methods · anatomy · the call stack · pass by value · arrays as parameters · ref and out · default and named arguments · overloading



Asst. Prof. Hasan Oğuz, PhD

Istanbul Okan University · Vocational School

2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

By the end of this week you can …

Explain why

Methods avoid copy-paste (DRY), split big problems, make testing and reuse easy.

Define a method

Return type, name, parameter list, body, return. Local functions at the bottom of Program.cs.

Trace a call

Arguments are copied into parameters; a new frame goes on the call stack; it is removed on return.

Pass parameters

By value (copy), arrays (reference copied), ref (alias), out (result).

Write flexible methods

Default values, named arguments and overloading (same name, different parameters).

Refactor and debug

Turn earlier programs into methods; step into calls with F11 in VS Code.

You have written the same code again and again

Until now: the same loop, copied for every input

```
Console.Write("Age: ");
int age;
while (!int.TryParse(Console.ReadLine(), out age))
{
    Console.Write("Try again: ");
}
Console.Write("Year: ");
int year;
while (!int.TryParse(Console.ReadLine(), out year))
{
    Console.Write("Try again: ");
}
```

Copy-paste = copied bugs. Fix one copy, forget the other.

Week 12: call a method

```
int age = ReadInt("Age: ");
int year = ReadInt("Year: ");
```

written ONCE (Example 10)

```
static int ReadInt(string prompt)
{
    Console.Write(prompt);
    int value;
    while (!int.TryParse(Console.ReadLine(), out value))
    {
        Console.Write("  Whole number, please: ");
    }
    return value;
}
```

A method = a named, reusable block of code.



PART 1

Defining and calling methods

Why methods · anatomy of a method · local functions in Program.cs · void and value-returning

Four reasons professionals split programs into methods

DRY

Write once, call many times; fix a bug in ONE place.

Decomposition

Split a big problem into small named steps.

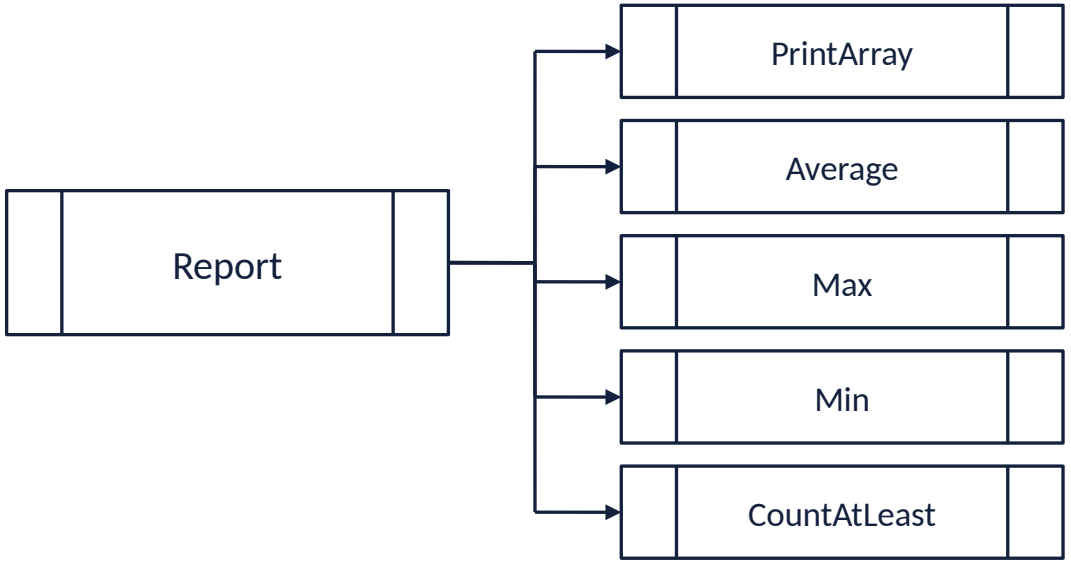
Testing

Check one small method with known inputs.

Reuse

Use the same method in many programs.

Structure chart: Example 11 (grade report)



Flowchart link (Week 2): the predefined-process box is a method call.

Draw a line: written once, called five times

Program.cs · the statements that run

```
PrintLine('=', 30);
Console.WriteLine("          OKAN CAFE");
PrintLine('=', 30);
PrintItem("Tea x2", 60);
PrintItem("Simit x1", 25);
PrintItem("Toast x1", 110);
PrintLine('-', 30);
PrintItem("TOTAL", 60 + 25 + 110);
PrintLine('=', 30);
PrintLine('*', 12);
```

Console

```
==============================
          OKAN CAFE
==============================
Tea x2                  60.00 TL
Simit x1                25.00 TL
Toast x1               110.00 TL
------------------------------
TOTAL                  195.00 TL
==============================
************
```

Week 6 style (no methods)

```
for (int i = 0; i < 30; i++)
{
    Console.Write('=');
}
Console.WriteLine();
// … and the same loop
// again for every line
```

Without methods: 5 lines of the receipt = 5 copies of the loop (25 lines).

With PrintLine: 1 loop + 5 one-line calls. Change the loop once, all lines change.

Same method, different arguments in every call

PrintLine: two parameters (char, int)

```
// prints symbol count times, then a new line
static void PrintLine(char symbol, int count)
{
    for (int i = 0; i < count; i++)
    {
        Console.Write(symbol);
    }
    Console.WriteLine();
}
```

PrintItem: two parameters (string, double)

```
// prints one receipt line (30 characters wide)
static void PrintItem(string name, double amount)
{
    Console.WriteLine($"{name,-18}{amount,9:F2} TL");
}
```

call	1st parameter	2nd parameter
`PrintLine('=', 30)`	symbol = '='	count = 30
`PrintLine('-', 30)`	symbol = '-'	count = 30
`PrintLine('*', 12)`	symbol = '*'	count = 12
`PrintItem("Tea x2", 60)`	name = "Tea x2"	amount = 60
`PrintItem("TOTAL", 60 + 25 + 110)`	name = "TOTAL"	**amount = 195**



Arguments are evaluated first: only the value 195 is copied into amount.

static int Square(int x) — five parts

`static`	`int`	`Square`	`(int x)`
modifier uses only its parameters	**return type** type of the result (void = none)	**name** PascalCase verb or noun	**parameter list** type + name of each input

the definition (at the bottom of Program.cs)

```
// returns x * x
static int Square(int x)
{
    return x * x;   // ends the method, sends the value back
}
```

calls (expressions)

```
int y = Square(5);        // 25
int z = Square(y) + 1;    // 626
Console.WriteLine(Square(3));
```

Parameter x

a variable that gets its value from the call

Argument 5

the value written in the call

Program.cs: statements first, local functions at the bottom

Program.cs

① **top-level statements**
run from top to bottom; the program starts here

```
int y = Square(5);
```

call

② **local functions (methods)**
run only when called; any order

```
// returns x * x
static int Square(int x) { … }
```

what the compiler builds (classic form)

```
class Program
{
    static void Main()
    {
        int y = Square(5);
    }
    static int Square(int x) { … }
}
```

static local function: cannot use main-program variables (CS8421). Pass them as parameters.

Four small methods: one void, three value-returning

void: does something

```
// prints a greeting; void = it returns nothing
static void Greet(string name)
{
    Console.WriteLine($"Hello, {name}! Welcome to MCMP113.");
}
```

returns a double

```
// returns the area of a circle with the given radius
static double CircleArea(double radius)
{
    return Math.PI * radius * radius;
}
```

return int values; Cube calls Square

```
// returns x * x
static int Square(int x)
{
    return x * x;
}

// returns x * x * x by calling Square
static int Cube(int x)
{
    return Square(x) * x;
}
```



You already call methods

Console.WriteLine, int.Parse, Math.Sqrt: written by Microsoft.

Calling: a statement, or a value inside an expression

Program.cs · top-level statements

```
Greet("Elif");
Greet("Mehmet");

Console.Write("Enter an integer: ");
int n = int.Parse(Console.ReadLine());
Console.WriteLine($"Square({n}) = {Square(n)}");
Console.WriteLine($"Cube({n}) = {Cube(n)}");

int sum = Square(3) + Square(4);  // calls in an expression
Console.WriteLine($"Square(3) + Square(4) = {sum}");

Console.Write("Radius in cm: ");
double r = double.Parse(Console.ReadLine());
double area = CircleArea(r);  // store the result
Console.WriteLine($"Area = {area:F2} cm^2");
```

Console

```
Hello, Elif! Welcome to MCMP113.
Hello, Mehmet! Welcome to MCMP113.
Enter an integer: 7
Square(7) = 49
Cube(7) = 343
Square(3) + Square(4) = 25
Radius in cm: 2.5
Area = 19.63 cm^2
```

call	kind	value
`Greet("Elif")`	statement	–
`Square(7)`	expression	49
`CircleArea(2.5)`	expression	19.634…

Does the method DO something or COMPUTE something?

	void method	value-returning method
header	`static void Greet(string name)`	`static int Square(int x)`
purpose	an action: print, change an array	a computation: gives a result
return	none (or return; to leave early)	return value; required
call	`Greet("Elif");`	`int y = Square(5);`
names	PrintLine, ShowMenu, Swap	Square, Average, IsPrime

void has no value

```
int n = Greet("Ali");
// error CS0029: Cannot implicitly convert
//               type 'void' to 'int'
```

every path must return

```
static int Sign(int x) { if (x > 0) { return 1; } }
// error CS0161: 'Sign(int)': not all code
//               paths return a value
```



PART 2

Calls and the call stack

What happens step by step · stack frames · parameters vs arguments

>_ A CALL, STEP BY STEP

double c = Hypotenuse(a, b); — six steps

1 **Evaluate the arguments**
a → 3, b → 4

2 **New frame, copy values**
x ← 3, y ← 4

3 **Jump into the method**
main waits at this line

4 **Run the body**
may call Square …

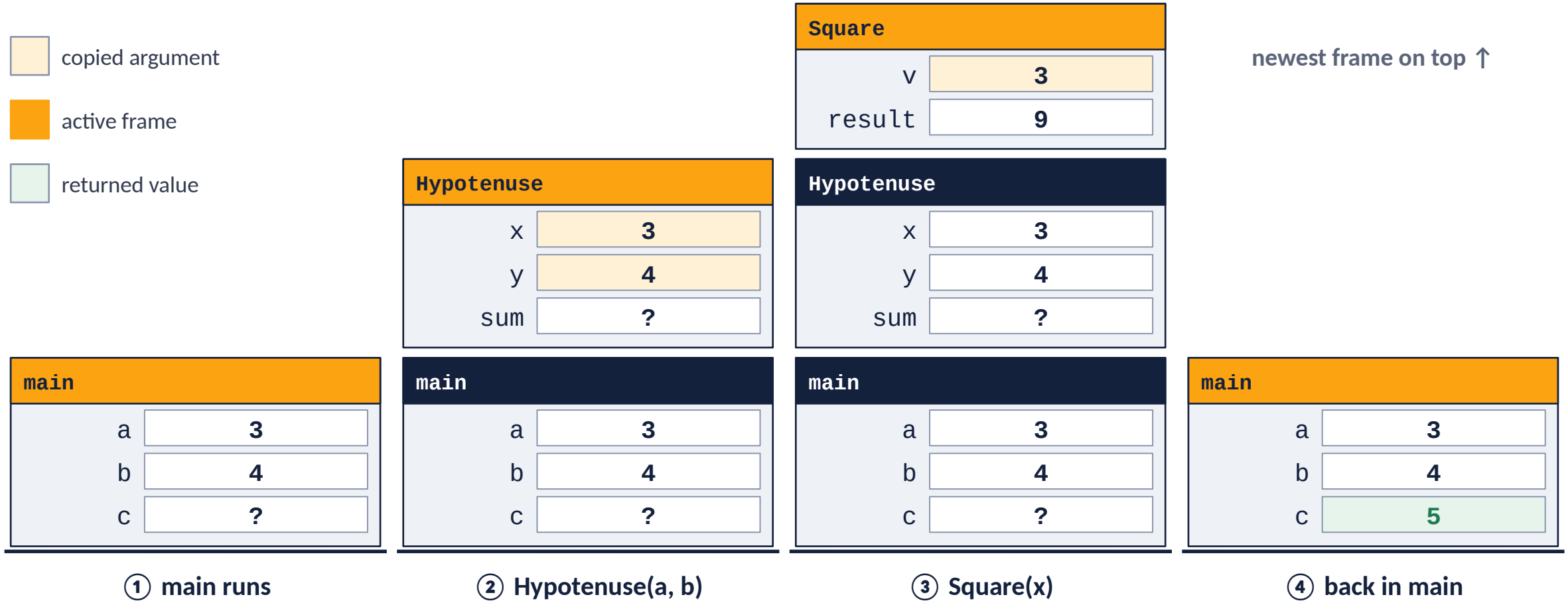
5 **return sends a value back**
Hypotenuse(a, b) → 5

6 **Frame removed**
x, y, sum, result are gone

Example 3 (without the trace lines)

```
static double Hypotenuse(double x, double y)
{
    double sum = Square(x) + Square(y);
    double result = Math.Sqrt(sum);
    return result;
}
```

Four snapshots of memory while Example 3 runs



A program that prints its own call stack

Program.cs (main and Square; Hypotenuse on the previous slides)

```
double a = 3;
double b = 4;
Console.WriteLine($"main: a = {a}, b = {b}");
double c = Hypotenuse(a, b);
Console.WriteLine($"main: c = {c}");

static double Square(double v)
{
    Console.WriteLine($"    Square starts: v = {v}");
    double result = v * v;
    Console.WriteLine($"    Square returns {result}");
    return result;
}
```

Console

```
main: a = 3, b = 4
  Hypotenuse starts: x = 3, y = 4
    Square starts: v = 3
    Square returns 9
    Square starts: v = 4
    Square returns 16
  Hypotenuse returns 5
main: c = 5
```



Indentation = depth. main → Hypotenuse → Square: at most 3 frames.

Arguments are matched to parameters by POSITION

```
double c = Hypotenuse(a, b);
```

arguments (values)

```
static double Hypotenuse(double x, double y)
```

parameters (variables)

Parameter = the box in the header.
Argument = the value you put in.

Rule	Example of a violation → real compiler message
same number	Square() → CS7036: There is no argument given that corresponds to the required parameter 'x' of 'Square(int)'
compatible types	Square("5") → CS1503: Argument 1: cannot convert from 'string' to 'int'
names may differ	argument a goes into parameter x (position counts)

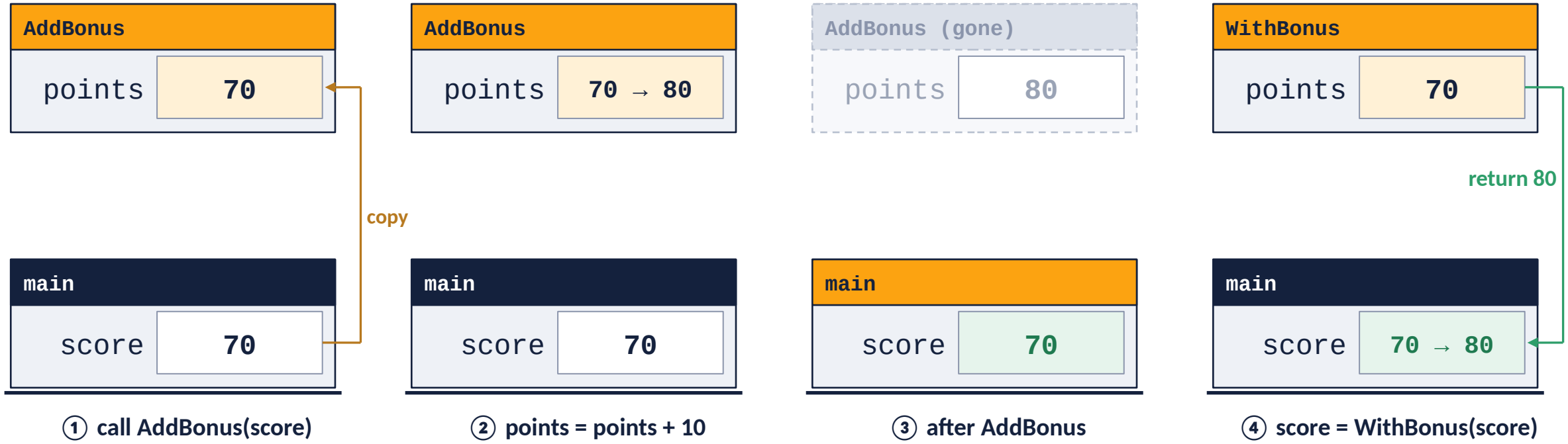


PART 3

Passing parameters

By value (a copy) · arrays (the reference is copied) · ref (an alias) · out (a result)

The parameter is a COPY: changing it does not change the caller



Real output: "inside AddBonus: points = 80" · "After AddBonus: score = 70" · "After WithBonus: score = 80"

AddBonus changes a copy; WithBonus returns the result

Program.cs (without the method comments)

```
int score = 70;
Console.WriteLine($"Before:            score = {score}");

AddBonus(score);           // 70 is copied into points
Console.WriteLine($"After AddBonus:    score = {score}");

score = WithBonus(score);  // store the returned value
Console.WriteLine($"After WithBonus:   score = {score}");

static void AddBonus(int points)
{
    points = points + 10;
    Console.WriteLine($"  inside AddBonus: points = {points}");
}

static int WithBonus(int points)
{
    return points + 10;
}
```

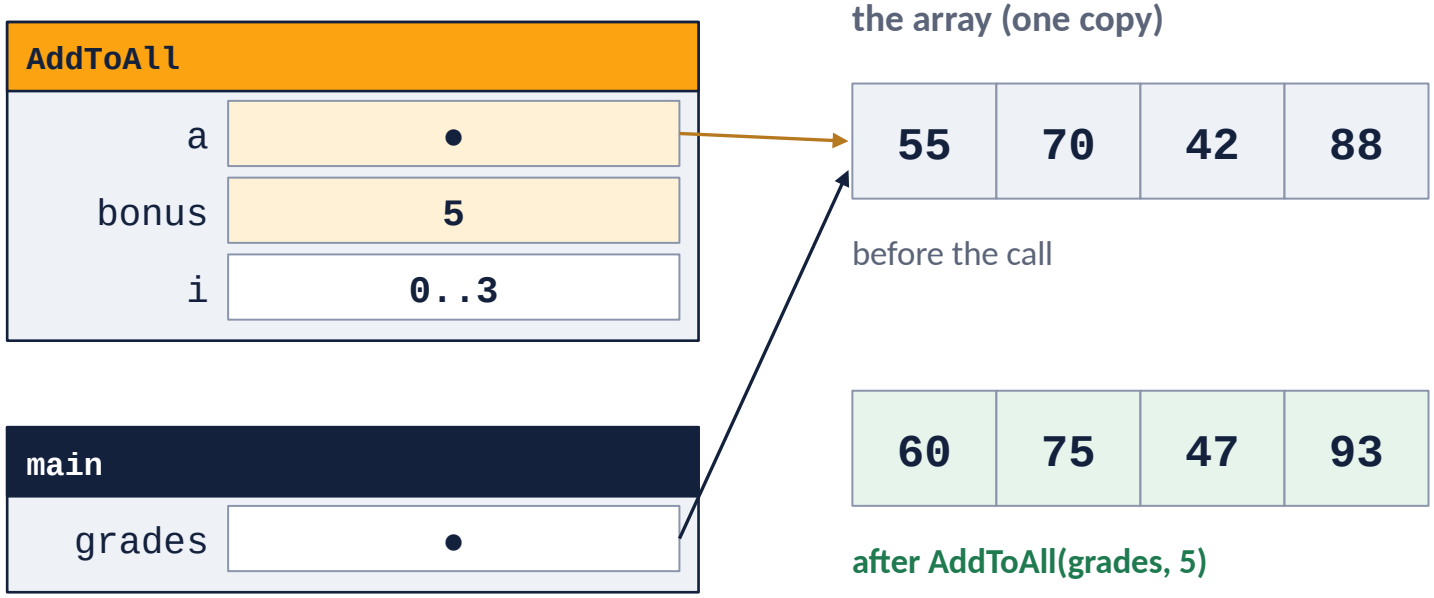
Console

```
Before:            score = 70
  inside AddBonus: points = 80
After AddBonus:    score = 70
After WithBonus:   score = 80
```

value parameter
int, double, bool, char, decimal …: a copy.

Need the new value?
return it, and assign it in the caller.

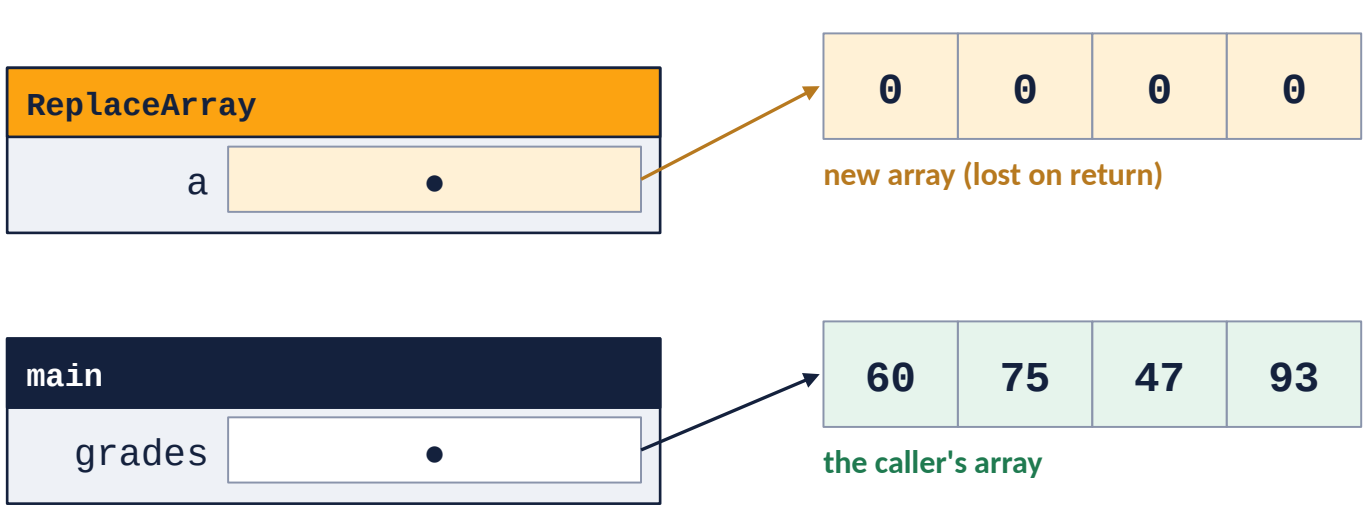
The REFERENCE is copied: both names point to the same array




```
changes the elements
static void AddToAll(int[] a, int bonus)
{
    for (int i = 0; i < a.Length; i++)
    {
        a[i] += bonus;
    }
}
```

Two arrows, one array: a[i] += bonus changes the elements the caller sees through grades.

But a NEW array in the parameter is not seen by the caller



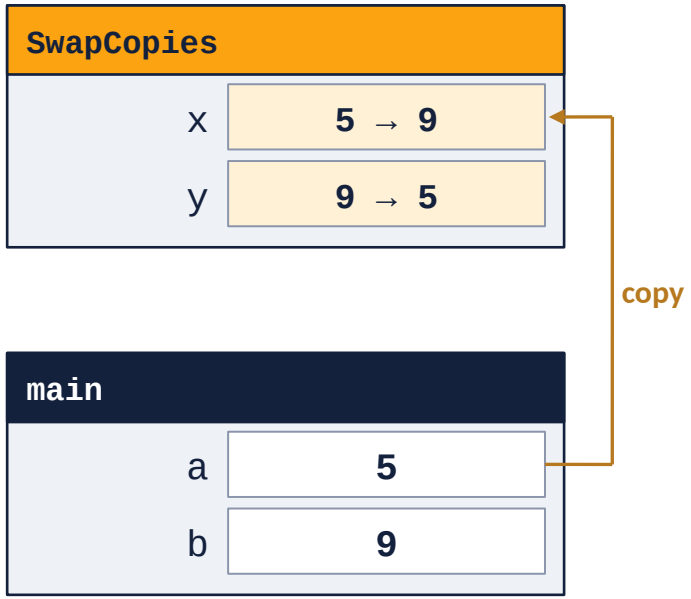

```
a = new … changes only the parameter
static void ReplaceArray(int[] a)
{
    a = new int[] { 0, 0, 0, 0 };
    PrintArray("  inside ReplaceArray", a);
}
```

```
Console
start                  : 55 70 42 88
after AddToAll         : 60 75 47 93
  inside ReplaceArray  : 0 0 0 0
after ReplaceArray     : 60 75 47 93
```

inside the method	caller sees it?
a[i] = …	yes
a = new int[…]	no

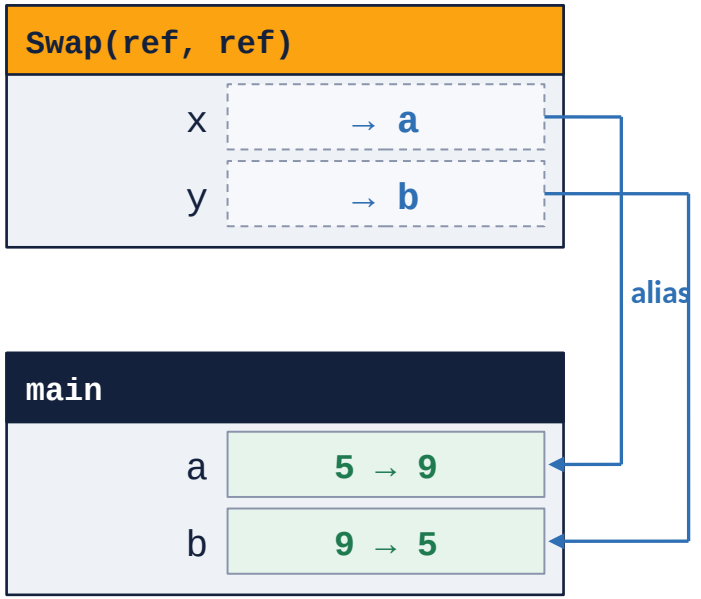
New array? Return it (Week 13).

ref: the parameter is an alias of the caller's variable



a = 5, b = 9: unchanged

```
SwapCopies(a, b);          // copies
Swap(ref a, ref b);        // aliases
```



a = 9, b = 5: swapped

```
Swap(a, b);    // error CS1620: Argument 1 must
               // be passed with the 'ref' keyword
```

`ref`
in the header AND in the call; argument must have a value.

>_ EXAMPLE 6 · 12_06_REFOUT

Swap with ref, Divide with out, and TryParse

```
two results through out (header split for the slide)
Divide(17, 5, out int q, out int r);
Console.WriteLine($"17 / 5: quotient {q}, remainder {r}");

static void Divide(int dividend, int divisor,
                   out int quotient, out int remainder)
{
    quotient = dividend / divisor;
    remainder = dividend % divisor;
}
```

```
// int.TryParse is an out method you already know (Week 5)
Console.Write("Your age: ");
if (int.TryParse(Console.ReadLine(), out int age))
{
    Console.WriteLine($"Next year you will be {age + 1}.");
}
else
```

Run 1

```
Start:            a = 5, b = 9
After SwapCopies: a = 5, b = 9
After Swap(ref):  a = 9, b = 5
17 / 5: quotient 3, remainder 2
Your age: 19
Next year you will be 20.
```

Run 2 (last lines)

```
Your age: nineteen
That is not a whole number.
```



out: the method MUST assign it (CS0177); the caller need not.

>_ FOUR WAYS TO PASS

Value, array, ref, out — what can the method change?

kind	header	call	what is passed	caller's variable can change?
value	`int x`	`F(n)`	a copy of the value	no
array	`int[] a`	`F(arr)`	a copy of the reference	elements yes, variable no
ref	`ref int x`	`F(ref n)`	the variable itself (alias); must have a value	yes
out	`out int x`	`F(out int n)`	the variable; the method must assign it	yes (always assigned)



Default choice
value parameters + return. Simple, safe, testable.



ref / out sparingly
Swap (ref); several results, TryParse (out).



PART 4

Flexible and reusable methods

Default and named arguments · overloading · refactoring earlier programs · debugging calls

Optional parameters and named arguments

calls: defaults are filled in, names pick the parameter

```
PrintLine();                       // both defaults: 20 times '-'
PrintLine('*');                    // count uses its default
PrintLine('=', 12);
PrintLine(count: 5);               // named: skip symbol
PrintLine(count: 8, symbol: '#');  // named: any order

Console.WriteLine($"Laptop: {Price(100m):F2} TL");
Console.WriteLine($"Bread:  {Price(100m, 0.01m):F2} TL");
Console.WriteLine($"Coat:   {Price(100m, discount: 0.25m):F2} TL");
Console.WriteLine($"Shoes:  {Price(net: 100m, vatRate: 0.10m, discount: 0.10m):F2} TL");
```

Console

```
----------
**********
==========
-----
########
Laptop: 120.00 TL
Bread:  101.00 TL
Coat:   90.00 TL
Shoes:  99.00 TL
```

the headers: default values after =

```
static void PrintLine(char symbol = '-', int count = 20)
static decimal Price(decimal net, decimal vatRate = 0.20m, decimal discount = 0m)
{
    decimal discounted = net * (1 - discount);
    return discounted * (1 + vatRate);
}
```

`decimal`
for money; literals need the m suffix: 0.20m

Overloading: one name, different parameter lists

Program.cs (methods in the Program class)

```
partial class Program
{
    // returns the larger of two ints
    static int Max(int a, int b)
    {
        return a > b ? a : b;
    }

    // returns the larger of two doubles
    static double Max(double a, double b)
    {
        return a > b ? a : b;
    }

    // returns the largest of three ints (reuses Max)
    static int Max(int a, int b, int c)
    {
        return Max(Max(a, b), c);
    }
```

Console

```
Max(3, 8)         = 8
Max(2.5, 1.75)    = 2.5
Max(4, 11, 7)     = 11
Max(scores)       = 93
Max(3, 2.5)       = 3
```

call	chosen version
`Max(3, 8)`	Max(int, int)
`Max(2.5, 1.75)`	Max(double, double)
`Max(scores)`	Max(int[])
`Max(3, 2.5)`	Max(double, double)

Local functions: no overloading (CS0128)

Refactoring: old algorithms become reusable methods

method	from	returns
`IsLeapYear(year)`	Week 5	bool
`IsPrime(n)`	Week 6	bool
`Factorial(n)`	Week 6	long
`Gcd(a, b)`	Weeks 1, 7	int
`Lcm(a, b)`	uses Gcd	int

Console

```
1900: not a leap year
2000: leap year
2024: leap year
2026: not a leap year
Primes up to 30: 2 3 5 7 11 13 17 19 23 29
5! = 120, 10! = 3628800, 20! = 2432902008176640000
Gcd(84, 36) = 12, Lcm(84, 36) = 252
Gcd(17, 5) = 1, Lcm(4, 6) = 12
```

IsPrime: a bool question

```
static bool IsPrime(int n)
{
    if (n < 2)
    {
        return false;
    }
    for (int d = 2; d * d <= n; d++)
    {
        if (n % d == 0)
        {
            return false;
        }
    }
    return true;
}
```

Input methods: the TryParse loop, written once

main: every input is one call

```
int n = ReadIntInRange("How many students (1-10)? ", 1, 10);
int[] grades = new int[n];
for (int i = 0; i < n; i++)
{
    grades[i] = ReadIntInRange($"Grade {i + 1} (0-100): ", 0, 100);
}
Console.WriteLine($"Grades: {string.Join(", ", grades)}");
Console.WriteLine($"Average: {Average(grades):F2}");
```

```
static int ReadIntInRange(string prompt, int min, int max)
{
    int value = ReadInt(prompt);
    while (value < min || value > max)
    {
        Console.WriteLine($"  Must be between {min} and {max}.");
        value = ReadInt(prompt);
    }
    return value;
}
```

Console

```
How many students (1-10)? abc
  Whole number, please: 12
  Must be between 1 and 10.
How many students (1-10)? 3
Grade 1 (0-100): 105
  Must be between 0 and 100.
Grade 1 (0-100): 70
Grade 2 (0-100): 85
Grade 3 (0-100): -5
  Must be between 0 and 100.
Grade 3 (0-100): 40
Grades: 70, 85, 40
Average: 65.00
```

An array toolkit: one Report, called for two groups

Report calls five methods

```
static void Report(string title, double[] grades)
{
    Console.WriteLine($"--- {title} ---");
    PrintArray(grades);
    Console.WriteLine($"Average: {Average(grades):F2}");
    double max = Max(grades);
    double min = Min(grades);
    Console.WriteLine($"Max: {max}, Min: {min}, range: {max - min}");
    int passed = CountAtLeast(grades, 50);
    Console.WriteLine($"Passed (>= 50): {passed} of {grades.Length}");
}
```

main: two groups, one Report

```
double[] groupA = { 72.5, 45, 88, 60, 95.5, 38 };
double[] groupB = { 80, 65, 90 };

Report("Group A", groupA);
Report("Group B", groupB);
```

Console

```
--- Group A ---
Grades: 72.5 45 88 60 95.5 38
Average: 66.50
Max: 95.5, Min: 38, range: 57.5
Passed (>= 50): 4 of 6
--- Group B ---
Grades: 80 65 90
Average: 78.33
Max: 90, Min: 65, range: 25
Passed (>= 50): 3 of 3
```

A menu program built from small methods

```
int choice;
do
{
    ShowMenu();
    choice = ReadIntInRange("Choice: ", 0, 5);
    if (choice != 0)
    {
        double x = ReadDouble("First number:  ");
        double y = ReadDouble("Second number: ");
        if (choice == 4 && y == 0)
        {
            Console.WriteLine("Error: division by zero.");
        }
        else
        {
            double result = Calculate(choice, x, y);
            Console.WriteLine($"Result: {result:F2}");
        }
    }
} while (choice != 0);
```

Calculate (switch expression)

```
static double Calculate(int op, double x, double y)
{
    return op switch
    {
        1 => x + y,
        2 => x - y,
        3 => x * y,
        4 => x / y,
        _ => Math.Pow(x, y),
    };
}
```

Console (third round of the run)

```
Choice: 7
  Please type 0-5: 5
First number:  2
Second number: 10
Result: 1024.00
```

Breakpoints, Step Into (F11) and the CALL STACK panel

```
5  int number = int.Parse(Console.ReadLine());
6
7  int count = CountDigits(number);
8  int sum = DigitSum(number);      ◀ breakpoint (F9)
9  int reversed = Reverse(number);
```

```
CALL STACK
after F11:  DigitSum  ▸  <Main>$
```

inside DigitSum(2026)	n % 10	sum	n after n /= 10
start	–	0	2026
1st pass	6	6	202
2nd pass	2	8	20
3rd pass	0	8	2
4th pass	2	10	0 → return 10

key	action
F9	toggle breakpoint
F5	start / continue
F10	Step Over (run the call)
F11	Step Into (enter it)
Shift+F11	Step Out (finish it)

Console

```
Enter a positive integer: 2026
Digits: 4, digit sum: 10, reversed: 6202
Average digit: 2.50
```

Method bugs and how to fix them

Mistake	Symptom	Fix
`Square() / Square("5")`	CS7036 / CS1503	right number and types of arguments
`int n = Greet("Ali");`	CS0029 (void has no value)	call a void method as a statement
`Swap(a, b);  // header has ref`	CS1620	Swap(ref a, ref b);
two local functions named Max	CS0128	different names, or methods in the class
`Square(n);  // result ignored`	no error, n unchanged	`n = Square(n);`
expecting points = 80 in the caller	value parameter: copy	return the value (or ref)
printing inside a "compute" method	cannot reuse the result	return it; print in the caller

>_ PREDICT THE OUTPUT

Quiz: what is printed?

Q1 · same name, two boxes

```
int x = 5;
Change(x);
Console.WriteLine(x);
x = Twice(x);
Console.WriteLine(x);

static void Change(int x)
{
    x = x * 100;
}

static int Twice(int n)
{
    return n * 2;
}
```

Q2 · array, value, ref

```
int[] a = { 1, 2, 3 };
int k = 10;
Update(a, k);
Console.WriteLine($"{a[0]} {k}");
Bump(ref k);
Console.WriteLine(k);

static void Update(int[] arr, int n)
{
    arr[0] = 99;
    n = 0;
}

static void Bump(ref int n)
{
    n++;
}
```

Q3 · order of calls

```
Console.WriteLine(F(2) + F(3));

static int F(int n)
{
    Console.Write($"F({n}) ");
    return n * n + 1;
}
```

Answers (checked by running the code)

Q1

```
5
10
```

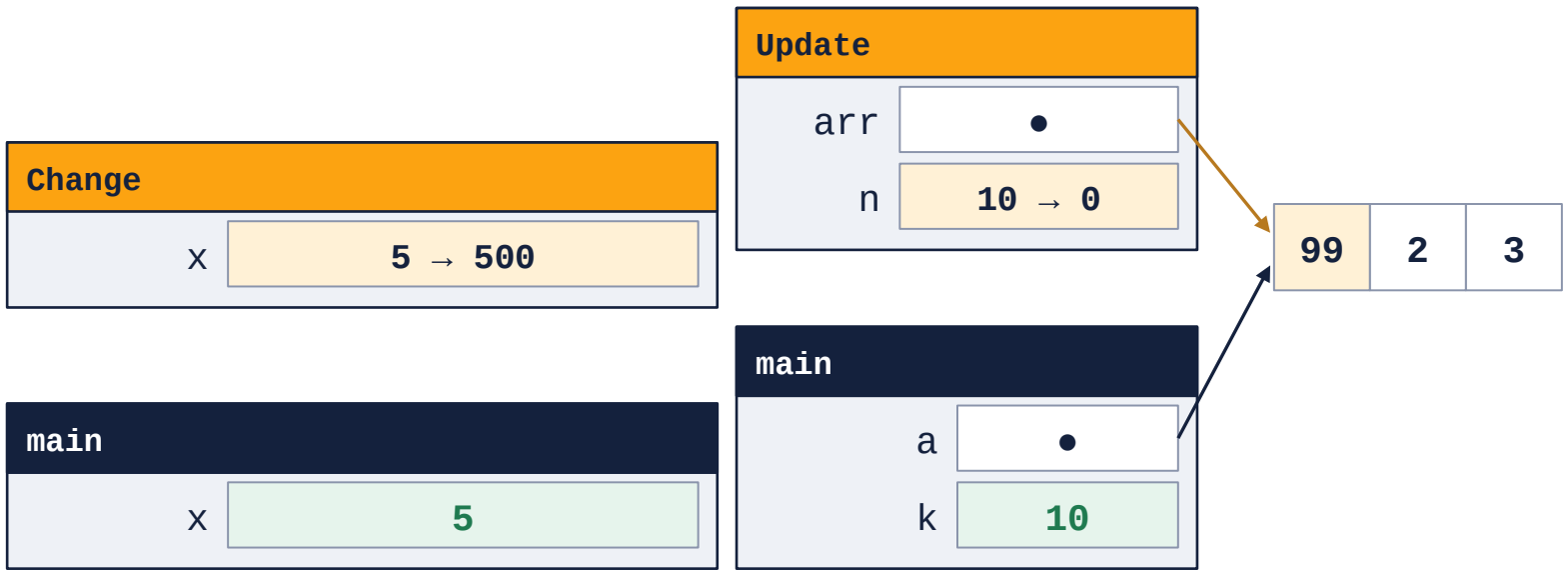
Q2

```
99 10
11
```

Q3

```
F(2) F(3) 15
```

Two different boxes called x:



call	prints	returns
F(2)	F(2)	5
F(3)	F(3)	10
5 + 10	15	–

Left to right: both calls print before WriteLine.

Your turn (pairs, 25 minutes)



E1 · Rectangle ★

Area(w, h) and Perimeter(w, h) return doubles. Read w and h, print both with F2.



E2 · Box of symbols ★

PrintBox(width, height, symbol = '#') prints a filled rectangle; reuse PrintLine from Example 1.



E3 · Temperature table ★★

CelsiusToFahrenheit(c) returns c · 9/5 + 32. Print a table for −10, 0, 10, …, 40 °C.



E4 · Sort three ★★

SortThree(ref a, ref b, ref c) puts three ints in ascending order using Swap(ref, ref).

>_ HOMEWORK

Before next week

H1 · Days in a month ★★

DaysInMonth(month, year) uses IsLeapYear. Print all 12 months of 2024 and the total (366).

H2 · Power ★

Power(x, exponent = 2) with a loop (no Math.Pow). Test Power(3), Power(2, 10), Power(1.5, 3).

H3 · Area overloads ★★

Area(r) circle, Area(w, h) rectangle, Area(a, b, c) triangle (Heron). Program class.

H4 · Sorting method ★★

BubbleSort(int[] a) (Week 10) and PrintArray(int[] a). Sort { 31, 12, 47, 8, 25, 19 }.

Submit the four Program.cs files on the course group; include one console run for each.

>_ SUMMARY

Key Takeaways

01 A method is a named block of code: write once, call many times (DRY).

02 Header: static, return type (or void), PascalCase name, parameter list. Local functions go at the bottom.

03 Each call gets a new stack frame; arguments are COPIED into parameters; return removes the frame.

04 Value parameters: the caller's variable never changes. Return the result instead.

05 Arrays: the reference is copied, so elements can change. ref = alias (Swap); out = extra results.

06 Defaults, named arguments and overloading make methods flexible. Debug calls with F11.

NEXT WEEK

Week 13: return values, void methods and scope

>_

MCMP113
Course materials